Reconstructing Source-Linked KPI Documentation: Architecture and Evaluation of LEAP

Meshaal Obaid Mouawad[a,*]

[a]*Department of Electrical and Computer Engineering, Mississippi State University, Box 9571, Mississippi State, 39762, MS, USA*

ARTICLE INFO

Keywords:
key performance indicators
software maintenance
program comprehension
literate programming
static analysis
traceability

ABSTRACT

Reconstructing documentation for a key performance indicator (KPI) requires recovering a named performance measure, its implemented calculation, and the evidence supporting its published business description. We present LEAP, a source-linked record architecture, and evaluate the reconstruction errors that remain when source relationships are preserved. Declared equations and expressions recovered from executable code occupy separate fields, alongside provenance and review information. A frozen 40-file challenge contains 65 reconciled KPI occurrences. Automatic promotion recovered six occurrences with two false positives, giving precision 0.750, recall 0.092 and F1 0.164; a structural baseline achieved F1 0.391. A reference-directed component review classified 54 cases: five correct, 30 partially correct, 12 incorrect and seven classified as no formula. Independent review of all eight automatically promoted pages found intact source links on every page and unsupported business-field presentation on every page. Source-level analysis connects these outcomes to expression selection, lost control conditions and publication defaults. A second frozen study characterizes occurrence matching and review workload on 355 Python files: the upgraded extractor produced 8,111 review candidates, matched 305 references under the primary rule and promoted none automatically. The study establishes a reproducible KPI reconstruction failure profile across occurrence identity, executable behavior and published field origins. It identifies concrete verification obligations for documentation tools that transform calculation code into business-facing records.

1. Introduction

A key performance indicator (KPI) is a named quantity used to assess an operational or business objective. A maintainer reconstructing its documentation must identify the relevant calculation, recover the behavior implemented by the program and explain the measure using supported business information. This study evaluates where those operations lose information in LEAP, a system that builds source-linked KPI records. On a frozen external challenge, all eight published pages retained their source links, the formula review classified five of 54 cases as correct, and every reviewed page presented unsupported business information. These results locate distinct reconstruction failures within an inspectable source-to-document pathway.

A percentage illustrates the difficulty. A program may return a production-to-input ratio for positive inputs and zero for an invalid denominator. A displayed ratio represents the main arithmetic expression; the guard and fallback determine additional behavior. The surrounding documentation may also identify an owner, reporting unit or approval state. Recovering the arithmetic, identifying a named performance measure and justifying those business fields require different evidence. An accurate file reference lets a reviewer inspect each assertion at its origin.

*Corresponding author
Email address: mm4922@msstate.edu (M.O. Mouawad)
ORCID(s): 0000-0002-1152-8324 (M.O. Mouawad)

Existing research provides substantial support for this task. Formula-documentation tools recover mathematical representations from programs [1, 2]. Semantic KPI approaches connect performance definitions to analytics [3, 4]. Documentation systems maintain links to code and signal changes that require review [5], while recent work in *Journal of Computer Languages* records development history and its contextual evidence [6]. These capabilities establish the foundations for source-linked reconstruction. The empirical question pursued here is which calculation and publication errors occur when an existing KPI implementation is transformed into a record intended for both technical and business inspection.

LEAP (Literate Programming for Automated KPI Extraction) retains a source-bound identity, candidate-local declarations, recovered executable expressions, business fields and review information in one record. Its declared and executable formula channels preserve their separate origins, including disagreement. The Interactive Bluebook renders selected records as linked KPI pages. This representation makes it possible to test the implemented transformation at several points and to trace an observed error back to the source, analyzer output or rendered field that produced it.

The evaluation has two stages with explicit reference objects. The primary external challenge contains 40 files and 65 reconciled KPI occurrences. Discovery is scored against these occurrences; a reference-directed component audit separately assesses 54 local formula reconstructions; and complete-page review covers all eight

automatically promoted pages. A second frozen study examines extraction output on 355 Python files from ten repositories, including the original and upgraded extractors and two baselines. It supplies strict native-coordinate matching and review-workload evidence, together with an inventory of retained expressions and review cards. Each endpoint has its own denominator and measurement procedure.

1.1. Research questions

RQ1 How does LEAP assemble source-bound KPI records, and which evidence origins and declared/executable disagreements survive publication?

RQ2 How do automatic promotion and review routing recover reference occurrences, and how does source-location granularity affect the measured discovery outcome?

RQ3 Which reconstruction mechanisms account for the original formula-review outcomes, and which expressions are retained in the second-stage extraction records?

RQ4 Which published fields are supported by their source evidence in the automatically promoted pages and the second-stage review cards?

1.2. Contributions

The first contribution is an implemented record architecture that keeps independently originating formula evidence attached to the same source occurrence through page generation. The second is an empirical characterization of KPI reconstruction failures, connecting source-level calculation behavior, independently recorded formula judgments and published field origins. The original challenge establishes the observed correctness distributions; the second stage characterizes review workload and the sensitivity of occurrence-based evaluation to output granularity. Together, the analyses explain which information remains inspectable and which transformations require additional checks before a generated KPI description can be treated as supported.

The evaluated original implementation is identified by commit `5c80321c9b20082e58ece26b06afdbc310f68850`; the upgraded extractor is identified by commit `289e7fda0ec48bc236c73deeace7e404a362d2e3`. Source snapshots, predictions, reference rows, scoring procedures and review outcomes are identified in the accompanying reproduction materials.

2. Background and Related Work

2.1. From literate programs to reconstructed explanations

Literate programming organizes executable instructions and their explanation as a connected work for readers [7]. Elucidative programming keeps separately stored source and narrative navigably linked [8]. Computational notebooks combine executable cells and prose, and their reproducibility depends on the relationships between recorded outputs and the computation that produced them [9]. Drasil generates coordinated code, mathematical descriptions and documentation from explicitly represented domain knowledge [10]. These approaches explain how shared information can support consistent artifacts.

The input to LEAP is an existing implementation whose calculation, declaration and business description may have been authored separately. Literate reconstruction here means assembling those existing representations into a connected explanation. Their origins remain explicit because a declared equation can disagree with the code. The research object is the fidelity of that reconstruction: which source behavior is recovered, which business assertions have support and which relationships survive rendering.

2.2. Formula recovery and program comprehension

Mathematical documentation requires translating program constructs into a representation that preserves their computational meaning. Moser et al.'s RbG combines static analysis and source annotations to produce formulas and decision tables [1]. Moseler et al. study formula code in Java, derive patterns for sums and products, and evaluate both detection and the correctness and completeness of recovered formulas [2]. Their work establishes separate evaluation of discovery and formula reconstruction. LEAP applies those concerns to named performance measures whose documentation also includes business fields and review status.

The source constructs matter to the recovered result. An arithmetic expression can depend on a preceding guard, local assignment, aggregate population or helper function. Slicito combines data-flow analysis with symbolic execution after a developer narrows the program scope, and its task-specific views connect domain objects to source methods [11]. This provides a recent example of making source analysis serve a concrete comprehension question. LEAP's operation-level description therefore distinguishes original expression selection from the upgraded analyzer's retained dependency and uncertainty information. Its evaluation records the resulting mathematical behavior and the information subsequently published.

2.3. KPI identity and semantic definitions

A calculation becomes a KPI through its relationship to a named performance objective. Identical arithmetic can serve a performance measure, a helper function or an intermediate value. Maté et al. derive analytical queries from semantic KPI specifications [3]; Roldán-García et al. provide ontology-driven KPI modeling and reasoning

[4]. MIKADO models KPI assessment for smart-city applications [12], and subsequent work studies consistency and propagation when KPI definitions evolve [13]. A recent survey synthesizes semantic models of performance indicators [14].

These approaches start from represented definitions and their relationships. Reconstructing a measure from source code requires first establishing which occurrence represents the named objective. Business-rule recovery provides related precedents: MARBLE recovers business-process descriptions from legacy software [15], and Chittimalli and Bhattacharyya use variable provenance and domain descriptions to construct business rules [16]. LEAP's reference rubric accordingly requires source evidence of a performance objective rather than treating arbitrary arithmetic as a positive instance. Its complete-page audit then examines whether the generated business fields are supported by their stated origins.

2.4. Source-linked documents and evidence provenance

Traceability recovery connects software documentation to source code [17]; provenance models distinguish the entities, activities and derivations involved in producing information [18]. Sodalite supports authoring and maintaining long-form code documentation, classifies link validity and flags documents for review. Horvath et al. explicitly identify comparison of document text against code as an additional validity question [5]. Thus link validity and content validity are established distinctions in documentation research.

Pham and Kelleher's *Code histories*, published in *Journal of Computer Languages*, records code changes, execution outputs and browser activity. Its observational evaluation compares captured events with screencast evidence [6]. The record supports inspection of the development process; its measured object is capture fidelity. LEAP examines a different transformation: deriving and publishing a performance measure from existing calculation code. The common methodological requirement is an explicit reference object against which retained information can be checked.

Policy-aware interface contracts bind declared obligations to observable evidence. Nguyen compiles obligations embedded in OpenAPI into tests of responses, state changes, audit events and endpoint lifecycle behavior [19]. This provides a specification-driven counterpart to LEAP's source-driven reconstruction: each published assertion requires an identified basis and an appropriate check.

2.5. Positioning the present evaluation

Table 1 compares the research inputs, scientific objects and evidence examined by the closest methods. These relationships are methodological: the studies evaluate different objects and reference populations. The present study connects source-occurrence identity, executable behavior and published business-field support within one reconstruction pathway. Its contribution is the recorded failure profile and its explanation through actual source constructs and output transformations.

This positioning determines the evaluation. Curated cases test evidence transport and disagreement preservation. An independently reconciled challenge tests occurrence discovery. Reference-directed formula review tests reconstruction even for source cases missed by discovery. Complete-page review tests what the selected outputs assert to readers. The larger second-stage study adds occurrence-matching and review-load observations under a separate frozen protocol. The following sections specify these operations before presenting their outcomes.

3. LEAP Architecture and Method

3.1. Reconstruction task and versions

The input is an existing source snapshot and any candidate-local declarations or metadata associated with it. The output is a set of KPI records whose fields retain their source origins, followed by a rendered page for each record selected for publication. Discovery chooses candidate regions; formula recovery derives an expression from a supplied region; page preparation combines the available information. The original implementation and the upgraded analyzer use different formula procedures, specified below. Their evaluated outputs are kept separate throughout the study.

3.2. Architectural overview

The reconstruction begins with an existing directory of source files. LEAP reads each eligible file as text, records its source location, and applies language-specific rules to locate possible KPI calculations or declarations. In the original implementation, candidate-promotion rules first select records for page preparation or retain them in Extraction Review. Page preparation then recovers the executable expression, prepares the declared formula separately, combines the available fields and assigns the displayed review status. Publication renders the prepared record as a navigable page. Figure 1 shows this original pathway and the evidence fields retained in the record.

The stages serve different purposes. Discovery identifies candidate regions, and promotion selects the page-preparation path. Mathematical reconstruction and field preparation then determine the evidence displayed for selected records. Page checks assign the displayed review status, and publication exposes the resulting fields and review reasons. The evaluation examines these distinct operations through occurrence discovery, formula review and complete-page assessment.

Table 1
Methodological comparison of pivotal predecessors and LEAP. Each row identifies the study's research object and evidence endpoint; the relationship column positions LEAP's evaluation.

Study	Research input	Output or scientific object	Evidence checked	Relationship to LEAP
RbG [1]	Scientific and engineering source with annotations	Formulas, decision tables and generated documentation	Tool demonstrations of extraction and document production	Establishes source-to-mathematics documentation; LEAP audits named KPI records and their fields.
Moseler et al. [2]	Java formula-code fragments	Detected sum/product code and reconstructed formulas	Detection against a manual oracle; formula correctness and completeness	Establishes separate assessment of discovery and reconstruction fidelity.
Maté et al. [3]	Semantic KPI definitions, strategic models and warehouse data	Derived MDX queries and KPI values in strategic views	Query derivation with initial theoretical and empirical evaluation	Starts from specified KPIs; LEAP starts from implemented calculations.
KPIOWL [4]	Business goals, measures, domain knowledge and water-management data	Ontology-based indicator selection and reasoning	Semantic conflicts in indicator relationships in a real case study	Establishes semantic KPI modeling; LEAP examines source-occurrence reconstruction.
Sodalite [5]	Authored long-form documents and linked code	Code-linked stories with link-validity status	Authoring use and out-of-date link classification	Establishes link/content validity as separate questions; LEAP audits formula and business-field support.
Slicito [11]	C# source and developer-selected analysis scope	Data-flow and symbolic-execution views	Industrial demonstration of asset-name validation	Provides scoped behavioral analysis; LEAP follows KPI reconstruction through publication.
Code histories [6]	IDE and browser activity during development	Detailed code-change and contextual history	Capture of programming activity in an observational study	Provides an evidence-capture precedent; LEAP checks reconstructed calculations and assertions.
LEAP (this study)	Existing calculation code, declarations and business metadata	Source-linked KPI records and role-facing pages	Occurrence coordinates, formula behavior and published field origins	Connects these endpoints through a source-grounded reconstruction failure analysis.

3.3. Candidate extraction, promotion, and deterministic source-derived identity

Once source files have been selected, the first analytical question is which regions might define a performance measure. LEAP walks the input directory in sorted path order, excludes configured binary and build locations, and chooses an extractor using the source format. It identifies possible KPI declarations and calculation regions, recording a candidate name, source path, line anchor, nearby code, and detection rule. These candidates are not yet validated KPI records. The next stage checks for evidence that the computation is a named performance measure rather than an arbitrary helper, status property, or intermediate value. Configured extraction rules select candidates for page preparation or retain them in Extraction Review. Page preparation later assigns the displayed review status. In the original implementation, promotion is a candidate-selection decision based on configured source cues;

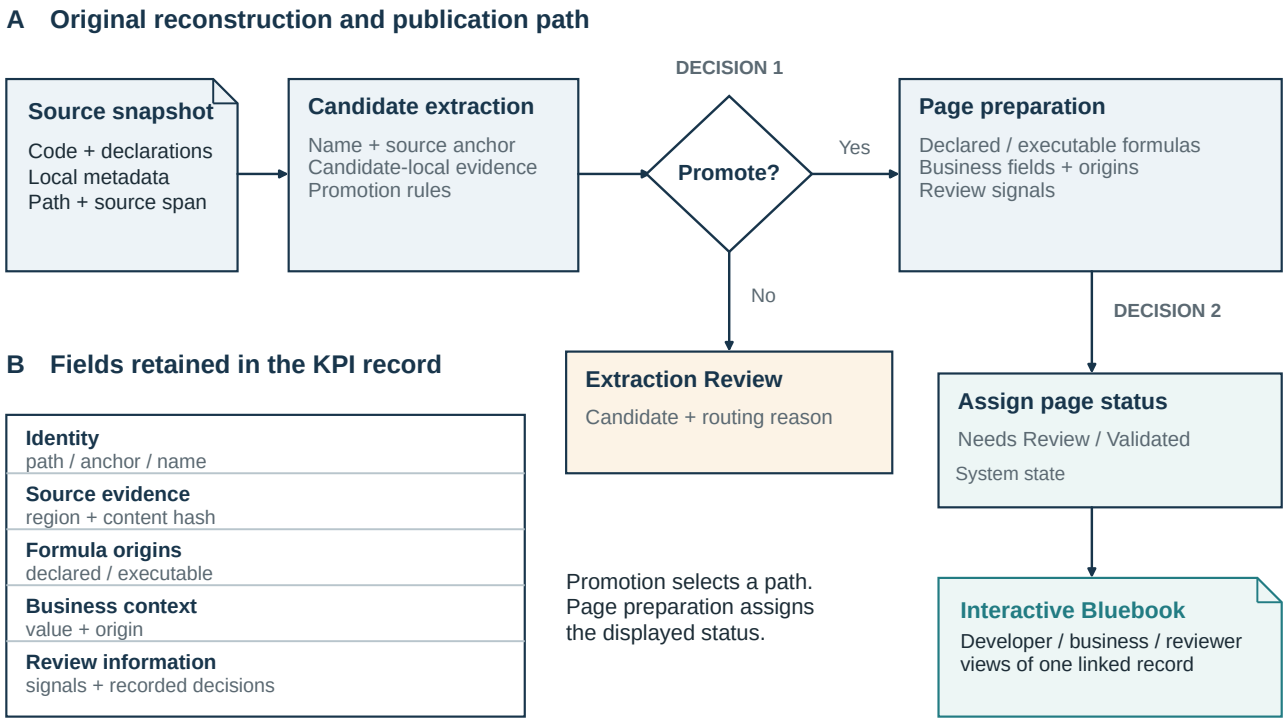


Figure 1: Original LEAP reconstruction and publication pathway. (A) Extraction promotion selects candidates for page preparation or retains them in Extraction Review. Page preparation constructs the formula representations and assigns the displayed system state. (B) The linked record retains identity, source evidence, formula origins, business context and review information. The developer, business and reviewer views read the same prepared record.

semantic and page-status checks occur later in the pathway.

One file may contain several performance measures, and the same KPI name may occur in different files. LEAP distinguishes those source-bound occurrences using the file path, the source-line anchor, and the KPI name. The path is made relative to the chosen source root, with forward slashes as separators. The anchor is converted to an integer and bounded below by one; an unavailable or invalid anchor becomes one. Name normalization trims surrounding whitespace and optional enclosing punctuation, collapses runs of underscores or whitespace into spaces, and then applies Unicode case folding. If no name is supplied, the fallback name is `KPI` before normalization.

The identifier hashes the relative path, anchor and normalized name. A separate source-content hash binds the captured evidence. Appendix A gives the exact serialization and its behavior under source edits.

Listing 1 summarizes the source-to-record selection path.

```
for each source file in deterministic path order:
    route text to the language/pattern extractor
    construct candidate records with source anchors
    attach candidate-local declared formula evidence
    for each candidate:
        apply promotion rules
        promote strong evidence or retain a review record
    collapse alternate detections around declarations
    retain independently named KPIs from the file
    assign deterministic source-derived record IDs
sort promoted records by source path, line, name, and ID
return promoted records and Extraction Review records
```

Listing 1: Source-to-record selection procedure.

3.4. Declared and executable formula reconstruction

A performance measure may have a documented equation and a separately implemented calculation. LEAP prepares declaration text in one channel and source-derived mathematical output in another. The original Mathematical Annotation and Tagging Engine (MATE) selects one expression from the source region supplied to it and converts that expression into mathematical presentation. The result contains the selected expression, its rendered HTML, a result name and an origin label.

The original executable mode removes comment text and, for Python-like input, blanks Python string tokens before applying ordered source-text patterns. These patterns cover SQL projections, DAX measure returns, assignments, Structured Text assignments and return statements. Assignment and return candidates are ranked by arithmetic-looking text and simple rejection rules. A small arithmetic parser renders names, numbers,

```
recover_formula(source_region, evidence_mode):
    if evidence_mode is executable:
        remove comment text
        blank STRING tokens in Python-like input
    select first accepted expression from ordered patterns:
        special formula / structured expression
        SQL projection / DAX return
        assignment / Structured Text assignment
        return / percentage or division fallback
    normalize the selected expression
    render supported arithmetic; otherwise use text fallback
    return expression, rendered formula, result name, origin
```

Listing 2: Original formula recovery from a supplied source region.

parentheses and arithmetic operators; residual text follows a fallback path. The function receives the source region and evidence mode, so expression selection is determined by the supplied text and rule order. Listing 2 specifies this operation.

The executable path is called separately from declared-formula preparation. Both outputs remain attached to the same KPI record, so a conflicting declaration can be displayed beside the source-derived expression. The original engine's selected expression is the object inspected in the reference-directed formula audit. Its treatment of executable strings, control conditions and multiple outputs is examined through the recorded cases in the results.

3.5. KPI record

The declared and executable formula channels are useful only when both refer to the same source occurrence. LEAP therefore creates a *KPI record*: stored information that identifies the candidate, retains the relevant source region, records the independently obtained formula representations, and attaches business context and review information. The record is assembled before publication. A *KPI page* is its reader-facing presentation in the Interactive Bluebook, generated from the stored record rather than being another field inside it.

Table 2 defines these fields by their origin and purpose. The record stores an absent expression explicitly and retains the origin of the formula evidence that is present. The table supplies the field-level objects inspected in the publication audit.

3.6. Extraction promotion, page status, and review

The original implementation makes two successive decisions. The extractor first applies candidate-promotion rules. A passing candidate enters the KPI-page preparation path; a failing candidate is retained in Extraction Review with its source and the reason for that route. An explicit KPI marker together with formula, executable-calculation, or business-metadata evidence can satisfy an original promotion rule. This stage determines which records enter page generation.

Page preparation then computes a separate review status from the prepared evidence. A declared/executable formula conflict, an audit or alarm signal, or low confidence can assign *Needs Review*; an accepted override or satisfied page conditions can assign *Validated*. The displayed labels are system states. A record that passed extraction can therefore remain in Needs Review after page preparation. Figure 1A separates these two decisions and their outputs.

The *Interactive Bluebook* presents three views of a selected KPI record. The developer view exposes source and calculation evidence, the business view presents the formula and business context, and the reviewer view shows provenance and review information. The shared record in Figure 1B supplies these linked reading tasks. The complete-record evaluation examines the correctness of the information presented in each role.

3.7. Upgraded analysis and candidate routing

The upgraded implementation uses source-anchored syntax analysis to retain the evidence surrounding a selected calculation. For Python, it parses the abstract syntax tree, selects a function or expression using candidate coordinates, and records source expressions, operands, operators, defaults, guarded return sites and call dependencies. For a single straight-line return, it attempts bounded substitution of local scalar assignments with checks on evaluation order and repeated work. Guarded returns remain recorded as separate paths; calls retain their names and source sites with unresolved-dependency reasons. The analyzer emits `COMPLETE`, `PARTIAL`, `UNRESOLVED` or `NO_FORMULA` as analysis states.

SQL uses a parsed projection and retains query context, guards, defaults and dependencies. DAX uses a bounded expression grammar with measure selection and review information for control/filter calls. Structured Text uses its parsed assignment and return structures. The measured external formula cases and the second-stage corpus are Python; the implementation inventory in Supplement S3 identifies the additional language routes and their code locations.

Candidate routing retains the available expression and its unresolved reasons in a review record. The evaluated upgraded extractor emitted review candidates without automatic promotions. Its renderer produces candidate review cards containing source association and analysis evidence. The second-stage study examines those native extractor records and cards; the original component-level formula procedure is evaluated separately on the original challenge.

3.8. Worked transformations

The following curated source examples illustrate the two formula channels and their publication. They are implementation demonstrations, distinct from the external challenge cases. The first uses a production

Table 2
Evidence fields retained in a KPI record and shown on its published KPI page.

Field	Origin	Purpose when published
Source-bound identity	Relative source path, candidate anchor, and normalized name	Distinguish KPI occurrences and link the page to its source.
Source evidence	Source file, source span, and captured code context	Show the implementation region supporting the record.
Declared formula	Formula stated in a candidate-local declaration or associated metadata	Identify the stated mathematical meaning and where it was declared.
Executable-formula evidence	Selected source expression and analysis of supported program behavior	Expose the recovered calculation and any unresolved semantic details.
Business context	Source assertions, associated metadata, or indicated inference	Display the intended measure and the origin of each business-facing field.
Review information	Formula comparisons, candidate routing, review decisions, and evidence checks	Show why a record requires inspection and which decisions have been recorded.

```python
# KPI: Ethylene Production Yield
# Formula: Ethylene Yield % =
#   (Ethylene Produced (tons) / Feedstock Input (tons)) * 100
# Unit: %
# Accountable Owner: Operations Data Owner

def calculate_ethylene_yield_pct(ethylene_produced_tons,
                                 feedstock_input_tons):
    if feedstock_input_tons <= 0:
        return 0.0
    return (ethylene_produced_tons / feedstock_input_tons) *
          100.0
```

Listing 3: Selected lines from the Ethylene Production Yield source example.

measure. Ethylene Production Yield measures the quantity of ethylene obtained relative to the feedstock consumed, expressed as a percentage. In the worked example, the numerator is ethylene production in tons and the denominator is feedstock input in tons. The documented calculation divides production by input and multiplies by 100. The Python source also supplies a guard: when feedstock input is nonpositive, it returns zero instead of evaluating that ratio. Listing 3 shows the declaration and implementation together:

LEAP constructs a source-bound record from this code and presents both the declared ratio and the executable evidence. The guard is part of the implemented behavior, so preserving only the positive-input ratio would not establish semantic fidelity. The curated publication checks verify that the record retains formula origins, source location, and review information; they are separate from the external source-semantic correctness measurements.

The yield example shows why a guard affects what a calculation means. A second example examines a different problem: the documented formula itself may disagree with the implemented expression. The Gross Margin Percentage case tests whether LEAP keeps those conflicting expressions separate. In this example, revenue denotes sales income, while cost denotes the related expense; the quantity is reported as a percentage. The declared business equation subtracts cost from revenue and divides by revenue:

$$\text{Gross Margin} = \frac{\text{Revenue} - \text{Cost}}{\text{Revenue}} \times 100, \tag{1}$$

The source implementation instead divides revenue by cost:

$$\text{Gross Margin}_{exec} = \frac{\text{Revenue}}{\text{Cost}} \times 100. \tag{2}$$

The evaluated implementation prepares these expressions through separate evidence channels, retains the source file `conflict_formula_margin.py`, assigns record ID `KPIREC-76E8A3DB97FA003A`, and publishes the record with *Needs Review*. This case directly tests whether a declared/executable disagreement survives reconstruction and publication. Figure 2 follows the two formula origins into the same published record.

4. Evaluation Method

4.1. Evaluation design and artifact identity

The architecture specifies what LEAP should preserve, and the worked cases show how those responsibilities appear in records. The evaluation next tests whether those properties hold in executed software and externally sourced calculations. The original evaluation uses the frozen implementation at commit `5c80321c9b20` and separates evidence into three scopes. The curated reconstruction scope contains 30 implementation files and 342 evaluated lines and exercises the complete source-to-record and publication path. A six-file public-source scope from three repositories provides bounded held-out evidence for deterministic reconstruction and record distinction. A separate external challenge benchmark contains 40 frozen public source files from four repositories. Experiment 2 reconciliation yields 65 resolved KPI instances used for scoring and two unresolved identities that are preserved outside the metric denominator. Table 3 summarizes these scopes.

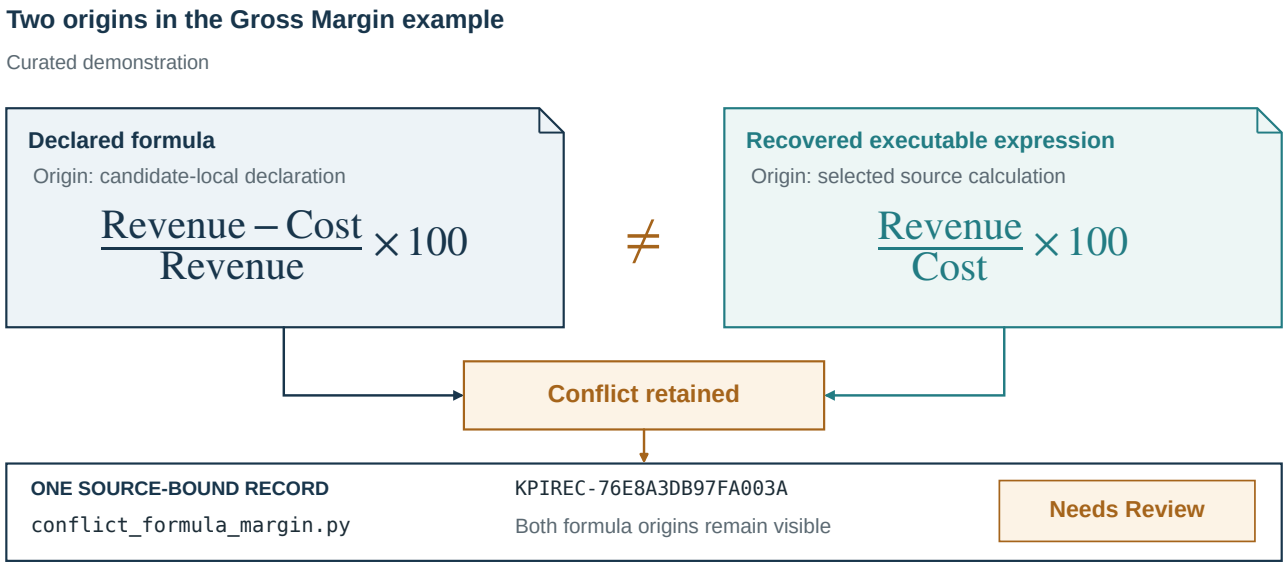


Figure 2: Two formula origins in the curated Gross Margin demonstration. The candidate-local declaration and recovered executable expression occupy separate fields of one source-bound record. The selected executable expression is the ratio from the source return; the source also contains a zero-cost fallback. The published record retains both origins and displays *Needs Review*.

Table 3
Evaluation scopes for architecture verification and external promotion measurement.

Scope	Files	Resolved KPIs	Promoted	Review-routed	Primary purpose
Curated demonstration	30	–	28	2	End-to-end evidence preservation and publication contracts
Six-file held-out source set	6	–	2	7	Deterministic reconstruction and same-name record distinction
External challenge corpus	40	65	8	33	Bounded promotion, baseline, scorer, repository, and review-channel diagnostics

4.2. Evaluation units and verification

RQ1 is tested through repeated record generation and publication contracts. The focused suite contains 14 regression tests for channel separation, identity, traversal and multi-KPI handling; 38 prepared-record fixtures exercise clean, conflicting and review-signaled cases. Twelve curated and five held-out publication checks inspect retained source and formula relationships. Appendix B records these checks and their execution outcomes.

RQ2 uses source-defined KPI occurrences as the discovery reference. RQ3 evaluates the original formula component on 54 reference-localized source regions and separately inventories expressions retained by the second-stage extraction adapters. RQ4 evaluates the eight automatically promoted pages and, separately, 300 matched review cards. The 54 formula cases are supplied calculation regions, so they are not a subset defined by successful discovery. All eight promoted pages are included in the page audit.

4.3. Held-out source protocol

The regression and prepared-record checks test defined behavior, but they do not show how the extractor treats source files outside the curated workspace. For an initial source-transfer check, the held-out package was frozen on 3 October 2026 from three public repositories and six source files:

- `rgcb01/manufacturing-oee-dashboard`, commit `b2673b41db22`: `src/oee_calculator.py`, `src/targets.py`, `src/scrap_analysis.py`, and `src/downtime_analysis.py`;
- `devv1310/Real-Time-Manufacturing-Dashboard`, commit `401252572629`: `utils.py`;
- `LourduAlex/machine_production_log`, commit `3f14246d1529`: `production/utils.py`.

The local held-out files were verified with `git hash-object` against their GitHub blob SHAs before execution. The package intentionally contains both direct scalar KPI calculations and helper/aggregation

code. This six-file scope is retained for deterministic reconstruction and record-distinction evidence; automatic discovery behavior is measured separately on the external challenge corpus below.

4.4. External challenge corpus, annotation, and baseline protocol

The six-file set tests reconstruction and record distinction on a small bounded input. Measuring KPI discovery requires a larger set with an independently defined reference population, so the next evaluation uses an external benchmark assembled as a *targeted challenge corpus*, not a prevalence sample. Before LEAP scoring, GitHub code search identified repositories containing naturally occurring KPI-like calculations such as OEE, gross margin, scrap, efficiency, quality, downtime, and related rates. After a search hit identified a repository, all eligible `.py` and `.sql` files at or below 12,000 bytes were included from a predeclared directory so that KPI-bearing files and neighboring non-KPI files remained in the sampling frame. The four frozen directories are `Samuel-TM/PwC_Audit_Case/src`, `flat-git/MIRW/src/metrics`, `messaoudismail6-byte/Smart_Industrial_Sorting_AIOT/analytics`, and `2711944586/NEXUS-PRIME/backend/app/models`. The corpus excludes LEAP development fixtures, the curated workspace, the earlier six-file held-out repositories, generated documentation, and author-created KPI demonstrations. It contains 39 Python files and one SQL file; the resolved KPI oracle is entirely Python.

The frozen annotation rubric defines a positive KPI instance as a source region that explicitly implements or declares a named operational or business performance measure with a calculable value used to assess performance, quality, efficiency, reliability, cost, throughput, yield, compliance, profitability, service performance, or another stated performance objective. Generic helpers, parsers, formatters, accessors, database fields, test utilities, visualization helpers, and arbitrary arithmetic remain negative unless the source itself establishes them as named performance measures. Formula normalization ignores only superficial syntax; guards, unit conversions, rounding, clipping, null behavior, aggregation semantics, and side effects remain material when they alter behavior.

Experiment 1 used the same 40 frozen files and criteria and is retained in the evidence package rather than overwritten. Its three-volunteer file-level Fleiss κ was 0.202, and provenance metadata required additional confirmation, so it is treated as exploratory replication evidence rather than the primary oracle. Experiment 2 was then defined as a confirmatory replication without changing the corpus or KPI definition. Three new volunteers completed source-only passes after attesting that they had not seen another volunteer's labels, Experiment 1 labels or results, prior annotations, LEAP predictions, baseline predictions, scores, or manuscript results. The volunteers reported roles of Data Analyst (5 years), Senior Software and Data Engineer/Code Reviewer (8 years), and audit/financial-analysis volunteer (0 years reported); all reported no relationship to LEAP development.

Before reconciliation, Experiment 2 pairwise file-level Cohen κ values were 0.533, 0.689, and 0.570. Fleiss κ across all three volunteers was 0.5958, with 32 of 40 file classifications unanimous. Exact source-plus-normalized-name candidate overlap was 14, 15, and 17 across the three pairs; only 10 candidate names were shared by all three and 26 by at least two. Strict agreement on source, normalized name, anchor, calculation bounds, kind, and review metadata was zero across all three pairs, making source-based reconciliation necessary rather than majority voting. Reconciliation preserved every submitted candidate, applied explicit split/merge decisions, retained 65 resolved KPI instances, left two EXT-013 risk identities unresolved, and produced 54 locally adjudicable executable-formula oracle rows plus 11 imported-call formula rows unresolved under the single-file source boundary.

Formula fidelity was evaluated separately after extraction scoring had been frozen. The capture pairs each of the 54 reference occurrences with its source snippet, adjudicated executable meaning, selected expression and rendered formula. The unchanged original executable-mode formula function reproduces all 54 expression strings and all 54 HTML strings when given these archived snippets. The 54 occurrence rows contain 35 distinct path/snippet inputs across 12 files. This is a reference-directed component audit: its source localization is supplied by the reference, independently of discovery success. Two volunteers reviewed these cases without seeing each other's judgments. Each case included the frozen source snippet and source URL, the adjudicated executable formula, and LEAP's recovered output. The primary verdict was one of `correct`, `partially_correct`, `incorrect`, `no_formula`, or `uncertain`. Five diagnostic dimensions recorded operands/operators, guards/exceptions, aggregation/window behavior, units/scaling, and rounding/null/default behavior. Exact primary-verdict agreement was 43/54 (79.6%) with Cohen $\kappa = 0.6035$. The 11 primary-verdict disagreements were then reconciled jointly from the same source evidence while the 43 agreeing verdicts remained frozen. The primary formula-review measure is final `correct`/54; a conditional measure excludes cases finally adjudicated `no_formula`. This category denotes a review judgment that the output is not a KPI formula; nonempty fragments can receive that label. The primary outcome is the frozen adjudicated classification, and the source-based diagnostic analysis examines the transformations

Table 4
Composition of the frozen Experiment 2 resolved scoring oracle. Two additional EXT-013 identities remain unresolved and are excluded from the metric denominator.

Repository	Files	Resolved positive files	Resolved KPI instances
`Samuel-TM/PwC_Audit_Case`	15	2	19
`flat-git/MIRW`	7	6	32
`messaoudismail6-byte/Smart_Industrial_Sorting_AIOT`	2	1	4
`2711944586/NEXUS-PRIME`	16	3	10
Total	40	12	65

underlying it. Dimension-level agreement remains a pre-reconciliation diagnostic because only the 11 primary-verdict disagreements received joint dimension reconciliation.

Complete-record fidelity was then evaluated on the full set of eight automatically promoted external KPI pages, after extraction and formula results had been frozen. Two additional volunteers independently inspected a static evidence package containing the HTML of each promoted KPI page, frozen source file, source excerpt around the reported anchor, record identifier, generated status, formula-origin note, review signals, Review Queue text, and record editing control. They attested that they had not seen the other volunteer's judgments, prior oracle labels, LEAP scores, baseline scores, or manuscript results. Ten component judgments covered source association, KPI identity support, source anchor, declared-formula origin, executable-evidence origin, business-metadata origin, review status/reason, provenance/lineage, published role separation, and review/edit controls; the overall verdict was `correct`, `partially_correct`, `incorrect`, or `uncertain`. A seven-step navigation checklist separately exercised the dashboard, KPI page links, Evidence & Lineage, Review Queue, record editor, static override snapshot, Discovery Workspace, and RACI Directory. The primary complete-record endpoint is the independently assigned overall verdict distribution; component disagreements are reported without forced consensus.

Table 4 reports the earlier challenge composition.

The primary comparison scores the *automatic promotion output* as positive predictions. Candidates sent to Extraction Review form a separate set, scored both alone and alongside promotions. A prediction is eligible to match a reference occurrence when the materialized paths are equal, the prediction anchor lies between three lines before the reference calculation start and three lines after its end, and the lowercase alphanumeric name tokens share at least one token. The executed original scorer visits predictions in their stored CSV order. For each prediction, it scans reference rows in their stored order, assigns the first still-unused eligible reference, and then stops searching for that prediction. Thus this is greedy one-to-one allocation in input order: it does not rank eligible references by exact-name equality or minimum anchor distance. Separate sensitivity analyses score zero-, one-, and three-line tolerances and alternative name rules; those variants do not replace the executed primary rule.

Two simple task-aligned baselines provide reference points. The keyword/regex baseline emits source lines containing predefined KPI terms. The structural baseline emits Python functions whose names contain KPI-like terms and whose bodies contain assignments or returns with arithmetic, comparisons, conditional expressions or calls. The 95% confidence intervals (CIs) for recall and F1 use 10,000 bootstrap resamples with the four repositories as sampling units and seed 20261003. Precision is reported as a point estimate; the supplementary verification record retains the precision resampling diagnostics, including draws with no predictions. Per-repository and leave-one-repository-out results are reported because four repository units generate wide uncertainty and uneven instance concentration.

4.5. Discovery measures

The source-labeling and reconciliation procedure establishes which KPI occurrences count as reference positives. We can therefore evaluate discovery by comparing each system's candidate identities with those occurrences. A true positive (TP) is a predicted KPI identity that matches one positive reference under the frozen source-coordinate and identity rule. A false positive (FP) is a predicted KPI identity that does not match a resolved positive reference. A false negative (FN) is a resolved positive reference with no accepted prediction. Precision describes the share of scored predictions matched under the specified rule, while recall describes the share of positive references matched. The F1 score combines precision and recall as their harmonic mean. With those terms defined, the three measures are:

$$\begin{aligned} \text{Precision} &= \frac{\text{TP}}{\text{TP}+\text{FP}}, \\ \text{Recall} &= \frac{\text{TP}}{\text{TP}+\text{FN}}, \\ \text{F1} &= \frac{2\text{TP}}{2\text{TP}+\text{FP}+\text{FN}}. \end{aligned} \tag{3}$$

Each denominator refers to the same frozen prediction channel and reference population. When a method emits no positive predictions, its precision has no denominator and is reported as undefined; recall and F1 are zero when the reference contains positive KPIs. Review-channel results are measured separately because a candidate sent for inspection is not an automatically accepted KPI.

4.6. Second-stage frozen source-reference evaluation

A separate prospective evaluation measures native candidate output against a strict occurrence-coordinate reference on a second frozen corpus. The earlier challenge supplies completed formula and eight-page correctness verdicts; the prospective study adds native-coordinate matching, review workload and separate formula-availability and review-card evidence assessments. Repository-first selection retained all eligible Python files in the selected directories of ten public repository families, producing 355 scored files and 30 context files. Repository commits and source hashes were frozen before system predictions. Repository keys R01–R10 resolve to repository URLs and frozen commits in S2, under `REVIEWER_REPRODUCTION/frozen_corpus/`, in each repository directory's `manifest.json`. The comparison identifies original commit `5c80321c9b20`, upgraded commit `289e7fda0ec4`, a keyword baseline, and a structural baseline, all applied to the same scored files. The original adapter invokes its candidate extractor as a standalone module and records fields present in the emitted candidates; it does not invoke the original page-preparation formula renderer. The upgraded adapter invokes the package extractor, which attaches native formula-evidence records. Consequently, the second-stage formula inventory measures retained extractor output at these executed boundaries. The keyword and structural baselines mark candidate function definitions, whereas many reference occurrences are assignments or returns within functions. The same scoring predicate is therefore applied to non-equivalent native candidate units. The endpoint measures compliance of native output with the reference occurrence coordinates after identity acceptance; function-level candidate identification and semantic KPI/formula correctness are separate assessment objects.

The adopted second-stage reference covers performance-related source occurrences more broadly than the original scalar KPI challenge. It includes numerical measures, categorical financial diagnoses, set-valued diagnostic outputs and test contracts; 40 of its 723 positive occurrences are in test source paths. For example, references include lists of affected buses and periods with an adverse current ratio. These entries retain their frozen source identities and are scored as the adopted reference population. The two studies are analyzed separately.

The initial reference annotations were produced by three separate AI annotators. Their metadata records source-only review and independently authored submissions; the reconciliation records retain those original submissions. The author approved all 38 documented reconciliation recommendations on 30 October 2026, including the recorded provenance qualifications, and the resulting reference was frozen. It contains 723 adopted positive source occurrences and 73 unresolved occurrences, distributed over 86 positive files, 265 negative files, and four unresolved files. KPI identity and formula semantics are recorded separately: 703 positive occurrences have unresolved formula semantics, and 20 have a confirmed source-local expression whose semantics still require approval.

Prediction matching requires an accepted identity decision, eligible source coordinates, and one-to-one allocation under the frozen scorer. On 31 October 2026, the author accepted all 1,627 populated identity recommendations: 465 MATCH and 1,162 NO_MATCH, together with the documented candidate/variant dispositions, eight unresolved-reference possibilities, and four corrections. The unchanged frozen scorer subsequently executed successfully. Accepted identity pairs were allocated one-to-one before counting true positives. Section 7 consolidates the remaining semantic and publication-correctness assessments. Table 5 keeps the two populations and their current evidence products separate.

Table 5
Frozen evaluation stages and their evidence populations.

Stage	Source population	Reference population	Evidence products
Original challenge	40 files, four repositories	65 resolved KPI occurrences; two unresolved	Scored predictions, formula verdicts, and eight page reviews.
Second-stage native-coordinate scoring	355 Python files, ten repositories	723 owner-approved positive occurrences; 73 unresolved	Strict native-coordinate matches, repository intervals and sensitivity, formula availability, and matched review-card readback.

4.7. Second-stage formula and card assessment

For each system, retained formula availability was bound to all 723 positive references. A missing match and a matched record without an expression both contribute to the no-formula population at this endpoint. Source-based recommendations were prepared for 1,446 system/reference cases and 300 matched initial review cards, then accepted by the author with unresolved cases retained. Seven axes covered operands/operators, guards/exceptions, aggregation/window behavior, units/scaling, rounding/clipping, null/default behavior and interprocedural dependencies. This second-stage instrument is distinct from the original five-dimension volunteer review.

Card assessment compared the actual rendered card and linked receipt with the accepted source association and frozen record. It recorded field origins, review reasons, provenance, wording and controls. Card-level classification and complete-page assessment were stored separately. The classifications and acceptance records are retained in S1; detailed second-stage outcomes appear in Appendix C.

4.8. Source-based diagnostic analysis

After the frozen evaluations, a supplementary diagnostic pass replayed the original formula function on the 54 archived snippets, traced its expression-selection rules and compared preserved outputs with their source regions. It summarized the original review dimensions without changing verdicts and selected cases that expose token removal, output selection and control-scope effects. These analyses use the existing observations; the source replay and tabulations are executable in S3.

A separate post hoc diagnostic examines the geometry of second-stage matching. It locates candidate function definitions, their owned return sites and enclosed reference occurrences. An exact normalized name and a single nonconstant return identify a restricted set of candidates whose definition anchor falls outside the reference return span. Constant fallback returns are allowed. This diagnostic is applied uniformly to all systems and reports source relationships separately from the frozen true-positive counts.

5. Results

The four questions follow the reconstruction objects: source-bound records, occurrence predictions, formula outputs and published fields. Curated verification and the original external challenge provide the primary evidence. The second-stage observations use the separately defined extraction and review-card endpoints.

5.1. RQ1: Record construction and evidence preservation

5.1.1. Curated reconstruction

The frozen artifact evaluated the 30-file curated workspace containing 342 implementation lines. It promoted 28 KPI records and routed two additional candidates to Extraction Review. Four repeated complete generations each produced 28 KPI pages and 32 RST pages. The complete RST publication tree had the same SHA-256 hash in all four runs: `c1fab3e76579` (SHA-256 prefix).

The Ethylene Production Yield path demonstrates the record fields defined in Table 2. The published source retains the KPI identity, source association, declared formula evidence, executable formula evidence, business metadata, review information, and publication structure. The curated publication contract found all 12 required evidence relationships in the generated Ethylene and Gross Margin KPI pages. These observations support the architectural claim that the source-to-record transformation carries the specified evidence roles into publication for the evaluated records.

5.1.2. Disagreement preservation

Having established that the curated records retain source and formula fields, the next test is whether a conflict between those fields survives publication. The Gross Margin Percentage case preserved two distinct mathematical representations in one published record. The declared evidence remained (Revenue − Cost)/Revenue × 100, while executable evidence remained (Revenue/Cost) × 100. The KPI page retained source file `conflict_formula_margin.py`, record ID `KPIREC-76E8A3DB97FA003A`, and review state *Needs Review*. All seven Gross Margin-specific publication checks passed within the 12-check curated publication contract.

The 14 focused regression tests passed in full and included direct assertions for independent declared/executable formula preparation, multi-KPI files, source-derived IDs, portable IDs across checkout roots, deterministic scan order, publication of both formula origins on the KPI page, and held-out source guardrails. The 38 prepared-record scenarios also passed in full. These checks establish that the implemented preparation and publication path preserves the tested evidence-origin distinctions and review signals.

5.1.3. Held-out identity and repeatability

The curated examples show record construction under controlled conditions. The next question is whether the same identity and publication path remains reproducible on previously unseen source. The six-file held-out package contained 339 implementation lines. LEAP promoted two direct OEE calculations and routed seven weaker candidates to Extraction Review. Both promoted records had the normalized name `calculate oee`, yet they remained separate because their relative source paths and anchors produced different source-derived identities:

- Record `KPIREC-381984E2BC3EEA6F`: source `devv13 10/Real-Time-Manufacturing-Dashboard/utils .py`;
- Record `KPIREC-884DC76258A7C9AD`: source `Lourdu Alex/machine_production_log/production/uti ls.py`.

Five held-out extraction scans produced the same canonical payload hash, `f6375456be32` (SHA-256 prefix). Three held-out publication runs each generated two KPI pages and six RST pages with the same complete RST-tree hash, `29d278820915` (SHA-256 prefix). The held-out publication-identity contract passed 5/5 checks.

5.2. RQ2: Occurrence discovery and review routing

5.2.1. Original challenge

The 40-file challenge measures occurrence recovery against the reconciled reference. All 40 external files materialized successfully and matched their frozen Git blob SHAs. The final Experiment 2 validation run used artifact commit `5c80321c9b20082e58ece2 6b06afdbc310f68850` with a clean Git state and passed the 14 focused regression tests and the prepared-record scenario harness before scoring. LEAP produced eight automatically promoted records and routed 33 additional candidates to Extraction Review.

Against the 65-instance resolved oracle, LEAP auto-promotion matched six instances and produced two unmatched promoted records, leaving 59 resolved instances unmatched. Table 6 reports the task-aligned comparison.

The structural baseline has both the highest point precision and the highest recall/F1 in Experiment 2. LEAP remains selective: its six true-positive automatic matches consist of four stocktake-variance measures and two financial ratios. The two false promotions are the generic `ratio` helper and the categorical `variance_type` property. The accepted-match audit found one unambiguous gold match for each of the six true positives. LEAP's TP/FP/FN values were unchanged at zero-, one-, and three-line tolerances and under shared-token versus exact-normalized name matching. The structural baseline was more scorer-sensitive: under exact-normalized names its F1 was 0.161, whereas shared-token scoring ranged from 0.345 at zero/one lines to 0.391 at three lines.

Repository-level outcomes were uneven. LEAP recovered 4/10 resolved instances in `NEXUS-PRIME` and 2/19 in `PwC_Audit_Case`, but none of the 32 `MIRW` or four `Smart_Industrial_Sorting_AIOT` instances. Leave-one-repository-out LEAP F1 ranged from 0.069 to 0.293; structural-baseline F1 ranged from 0.286 to 0.419. Of LEAP's 59 false negatives, 55 carry a technical-review requirement in the reconciled oracle: 44 are associated with zero/invalid-input guard semantics and 11 with insufficient local evidence; four have no technical-review reason. These reference characteristics describe the semantic context of the missed occurrences.

Table 7 retains the separate earlier review-channel operating points.

Extraction Review recovered three additional gold instances: a downtime-ratio function, aggregate availability within an OEE summary, and an OEE calculation routed from a broader function. It also contributed 30 unmatched review candidates. Thus the current two-channel workflow increases recall from 0.092 to 0.138 but reduces precision from 0.750 to 0.220 and raises F1 from 0.164 to 0.170. This quantifies the additional candidate load: three extra reference matches accompany 30 extra unmatched candidates.

5.2.2. Second-stage native-coordinate matching and review workload

Original LEAP emitted 19 automatic and 354 review candidates. The upgraded extractor emitted 8,111 review candidates and no automatic records; four candidates affected by unresolved reference regions were excluded from primary scoring, leaving 8,107. Under the frozen anchor-in-span and accepted-identity rule, the original combined channel matched three of 723 positive references and the upgraded review channel matched 305. The keyword and structural baselines emitted 96 and 91 candidates and had zero matches under this rule. Table 8 reports these native-coordinate operating points. Its common identity and coordinate predicates act on non-equivalent candidate units: the baselines emit function-definition anchors, whereas the references often locate calculations within those functions.

The upgraded result corresponds to 7,802 unmatched scored review candidates alongside 305 matches. R01

Table 6
Experiment 2 automatic-promotion performance on 65 resolved KPI instances. Recall and F1 intervals resample the four repositories 10,000 times; precision is a point estimate.

Method	TP	FP	FN	Precision	Recall (95% CI)	F1 (95% CI)
LEAP auto-promotion	6	2	59	0.750	0.092 [0.000, 0.286]	0.164 [0.000, 0.421]
Keyword/regex baseline	3	15	62	0.167	0.046 [0.000, 0.114]	0.072 [0.000, 0.139]
Structural baseline	17	5	48	0.773	0.262 [0.129, 0.330]	0.391 [0.200, 0.471]

Table 7
Review-channel operating points on the 65-instance Experiment 2 oracle.

Channel	Candidates	Precision	Recall	F1
Promoted only	8	0.750	0.092	0.164
Review only	33	0.091	0.046	0.061
Combined unique	41	0.220	0.138	0.170

contains 509 of the 723 positive references and contributes 258 upgraded matches. The upgraded-minus-original combined F1 difference under this strict endpoint is 6.361 percentage points, with paired repository-bootstrap 95% interval [-0.046, 7.286]. The predefined automatic-improvement criterion under the same endpoint was unmet: the upgraded automatic channel has no predictions, zero recall and F1, and undefined precision. Repository-level counts, all matching variants and unresolved-region scenarios are preserved in Appendix C.

5.2.3. Source-location granularity diagnostic

The zero baseline result depends on the occurrence-level localization rule. In R10, `financialHealthToolkit/CompaniesMod.py` defines `current_ratio` at line 78 and returns its calculation at line 80. Both baselines mark line 78; reference P1P00759 spans line 80. The definition anchor therefore falls outside the accepted reference span even though the function contains the ratio calculation. The original 40-file study used a different rule, allowing a three-line margin around the calculation span. Figure 3 shows the source locations and coordinate test for this second-stage example.

The uniform post hoc diagnostic found 30 function-anchored predictions in each baseline whose function bodies contain 51 distinct reference occurrences. A stricter syntactic screen finds 14 candidates per baseline with an exact normalized function/return-reference name and one owned nonconstant return, with optional constant fallbacks; 13 are guarded financial methods in R10. The same screen finds 20 original and 55 upgraded candidates. Table 9 reports these geometric relationships. These containment and name/return counts remain separate from the frozen TP/FP/FN and from semantic KPI/formula correctness judgments; the frozen identity decisions and all primary scores remain unchanged.

5.3. RQ3: Formula reconstruction

5.3.1. Original review classifications

The component audit contains 54 reference-localized cases. Eleven additional imported-call cases remain outside this local audit. The final review assigns seven of the 54 outputs to `no_formula`. Independent formula review produced exact overall-verdict agreement on 43/54 cases (79.6%, Cohen $\kappa = 0.6035$); the 11 remaining verdict disagreements were reconciled jointly from source evidence. Table 10 reports the final semantic verdict distribution.

The adjudicated correct-classification fraction is 5/54 (9.3%). Among the 47 cases not classified as `no_formula`, the correct-classification fraction is 5/47 (10.6%). Thirty-five of 54 cases (64.8%) are at least partially faithful, but that broader category is descriptive and is not treated as the correct-classification fraction. No case remains uncertain after reconciliation. The five cases classified correct are concentrated in the `NEXUS-PRIME` purchase and stocktake sources. The source-based analysis below explains the observed outputs at the level of the executed reconstruction rules.

5.3.2. Observed reconstruction mechanisms

The independent dimension records show recurring behavioral omissions. Both reviewers selected `no` for guard/exception preservation in 28 of 54 cases and for aggregation/window preservation in 21. Twenty-two of the 28 guard cases contain the relevant controls within the supplied snippet; the other six call a helper whose exception appears in the source oracle. All 21 jointly identified aggregation losses occur in snippets containing grouping and aggregate operations. Table 11 retains the original pairwise judgments, including their disagreement rates.

The unchanged executable-mode routine reproduces every archived expression and rendered formula. Its trace identifies three concrete mechanisms. First, removing Python string tokens also removes executable selectors. For EXP2-GOLD-049, the selected intermediate `runs_df["actual_output"].sum()` becomes `runs_df[].sum()`; the final scrap-to-output ratio is not selected. In EXP2-GOLD-020, removing a string assigned to a column selector allows the assignment pattern to consume the following line and return `else:`.

Table 8
Second-stage native output, strict occurrence-coordinate scoring. TP/FP/FN apply the frozen identity and coordinate predicates to non-equivalent native candidate units. Precision, recall and F1 are percentages. Combined means automatic plus review output; the upgraded combined output equals its review output.

System	Channel	TP	FP	FN	Precision	Recall	F1
Original LEAP	Automatic	1	18	722	5.263	0.138	0.270
Original LEAP	Review	2	352	721	0.565	0.277	0.371
Original LEAP	Combined	3	370	720	0.804	0.415	0.547
Upgraded LEAP	Review	305	7802	418	3.762	42.185	6.908
Keyword/regex	Automatic	0	96	723	0.000	0.000	0.000
Structural	Automatic	0	91	723	0.000	0.000	0.000

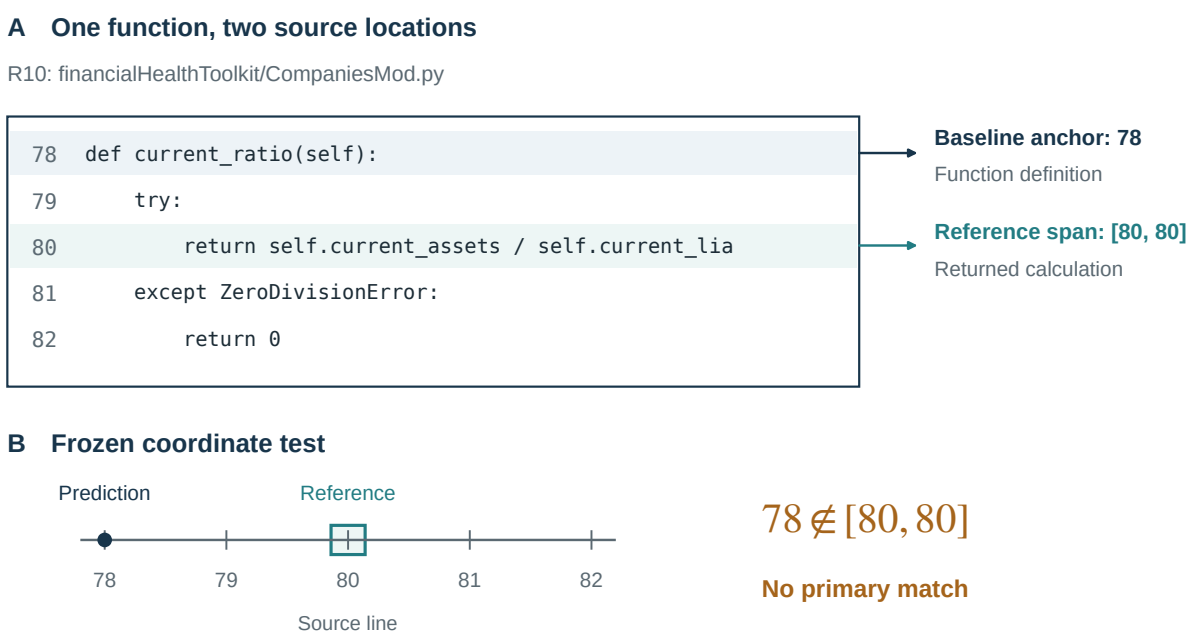


Figure 3: Source-location granularity in the second-stage frozen study. (A) Both baselines anchor `current_ratio` at its definition on line 78; reference P1P00759 occupies the returned calculation on line 80. The displayed source excerpt includes the zero-division fallback. (B) The primary anchor-in-span rule excludes line 78 from the reference interval [80, 80]. The enclosing function contains the reference return.

Table 9
Post hoc function-location diagnostic. Counts concern source relationships, separately from the frozen match verdicts.

System	Functions	References	Named returns
Original	36	54	20
Upgraded	179	537	55
Keyword	30	51	14
Structural	30	51	14

Table 10
Final reconciled semantic formula fidelity on 54 evaluated local cases, including the no-formula classification.

Final verdict	**Cases**	**Share**
Correct	5	9.3%
Partially correct	30	55.6%
Incorrect	12	22.2%
No formula classification	7	13.0%
Total	54	100.0%

Second, the formula routine selects an expression without receiving the requested output identity. EXP2-GOLD-041–044 supply the same source region containing availability, performance, quality and OEE. All four receive `runtime / planned`, the availability calculation. The review classifies this output as partial for availability and incorrect for the other three measures. A source link to the correct region therefore coexists with selection of the wrong result inside that region.

Table 11
Original formula-dimension judgments before reconciliation. Joint yes/no counts require the same explicit response from both reviewers.

Dimension	Agreement	Both yes	Both no
Operands/operators	26/54	17	9
Guards/exceptions	32/54	3	28
Aggregation/window	45/54	2	21
Units/scaling	35/54	4	0
Rounding, null/default	16/54	2	7

Third, expression selection preserves inline conditions more readily than conditions in preceding statements. EXP2-GOLD-059 retains the complete on-time-rate return, including its inline conditional and fallback, and is classified correct. EXP2-GOLD-061 selects the rounded stocktake percentage from a later return and omits the earlier zero-denominator branch, producing a partial classification. These are recorded examples of how statement layout affects the extracted representation. Figure 4 illustrates the three transformations using archived source and captured native-expression evidence. Table 12 links these diagnoses to the archived case identifiers and frozen review labels.

Native-expression text and rendered mathematics also differ. EXP2-GOLD-014 retains the call `ratio(revenue - cost, revenue)` as native text, while its captured HTML displays the single label `Ratio`. The formula-review workbook evaluates the native expression against its source oracle. The reproduced HTML supplies an additional presentation observation, preserved separately in S3. The capture has 53 nonempty expression strings; the seven no-formula classifications include one empty string and six nonempty fragments. Thus the conditional denominator of 47 is the complement of a review label, rather than a count of syntactically usable expressions.

5.3.3. Second-stage extractor-output availability

The original extraction adapter retains no formula in its 723 bound cases. The upgraded adapter has 418 reference occurrences unmatched under the primary rule, 208 matched records without an expression and 97 matched native expressions. The accepted classifications are therefore 723 original `NO_FORMULA`, and 626 upgraded `NO_FORMULA` plus 97 `UNRESOLVED`. These observations answer the output-availability part of RQ3 at the adapter boundary specified in the method. The original 54-case component audit uses a different invocation and retains its separate classifications. All 97 upgraded expressions remain semantically unresolved in the accepted second-stage record.

5.4. RQ4: Published field support

5.4.1. Eight automatically promoted pages

Both complete-record volunteers returned judgments on all eight KPI pages and both blinding attestations. Their overall record-fidelity verdicts agreed on all eight KPI pages (100% exact agreement; Cohen $\kappa = 1.0$): six KPI pages were `partially_correct`, two were `incorrect`, and none was fully correct or uncertain. The two incorrect records were the same two unmatched automatic promotions identified by extraction scoring: `Variance Type`, a categorical classification property, and the generic `Ratio` helper. The six accepted automatic KPI matches were all judged partially correct rather than incorrect.

Table 13 summarizes the earlier component-level page judgments.

The two reviewers agree on 79 of 80 page-by-component judgments (98.8%); the sole subfield disagreement is the KPI-identity judgment for the generic `Ratio` helper, and no consensus value is imputed for that field. Both volunteers independently passed all seven navigation checks. The source-link and evidence-role retention tests passed for the named source and origin fields in the eight reviewed pages. The separate formula audit classified five of 54 cases as correct; business-field origin accuracy failed in all eight pages; and complete-page correctness passed in none. These outcomes refer to different properties of the same reconstruction pipeline. All eight pages displayed `Needs Review` and exposed a record-editor path, yet unsupported metadata and authoritative presentation remained visible.

5.4.2. Second-stage review cards

The initial upgraded rendering run produced 7,873 cards, 196 failures and 42 outputs available only after a separate diagnostic rerun. Of the 305 matched upgraded records, 300 had an initially successful card and five failed rendering. The original system's three matched records had no available publication HTML. The 300 inspected cards preserve the expected source associations and anchors. Author-adjudicated assessment classifies each as partially correct within card scope and retains every complete-page outcome as unresolved. Business-field origin is unresolved on all 300; provenance and review/correction controls are partial. Appendix C retains all field-level outcomes and rendering dispositions.

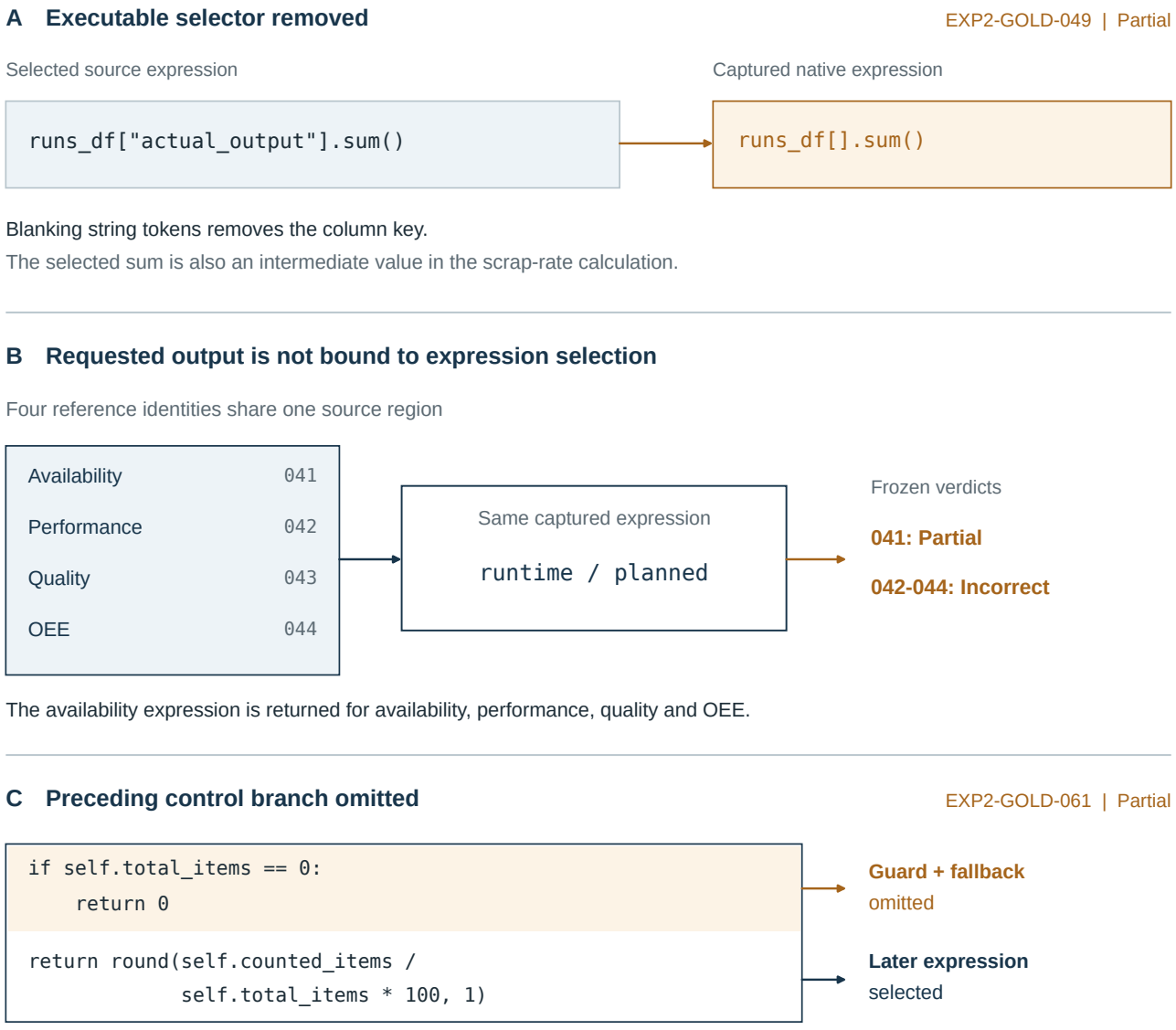


Figure 4: Observed reconstruction mechanisms in the original 54-case formula audit. (A) EXP2-GOLD-049 loses an executable column selector and retains an intermediate sum. (B) The shared source region for EXP2-GOLD-041–044 yields the availability expression for all four requested KPI identities. (C) EXP2-GOLD-061 retains the later rounded expression and omits the preceding zero-denominator branch; the return expression is line-wrapped for display. Labels reproduce the frozen native-expression verdicts: Partial denotes partially correct.

Table 12
Source-based diagnoses of preserved original formula outcomes. Case identifiers refer to the 54-row capture and review records in S3.

Case suffix	Source operation	Captured expression	Observed transformation	Frozen label
004	Indexed net profit divided by indexed revenue	`/ revenue`	Scalar-division fallback loses the indexed numerator and year selection.	Partial
020	Group and sum downtime by a chosen category	`else:`	String removal changes assignment context; the selector consumes a branch marker.	Incorrect
041	Guarded availability ratio	`runtime / planned`	Main arithmetic retained; guards, rounding and fallback omitted.	Partial
042–044	Performance, quality and OEE in the same source region	`runtime / planned`	Single-expression selection returns another output's calculation.	Incorrect
049	Guarded ratio of summed scrap to actual output	`runs_df[].sum()`	Executable column key removed; an intermediate sum selected.	Partial
059	Rounded percentage with inline conditional	Complete one-line conditional return	Inline default and arithmetic retained in native expression.	Correct
061	Zero guard followed by rounded percentage	Rounded percentage return	Preceding control branch omitted from selected expression.	Partial

Table 13
Independent component-level complete-record fidelity across the eight automatically promoted external KPI pages. Counts show exact inter-volunteer agreement and, where unanimous, the agreed positive outcome.

Record component	**Agreement**	**Agreed positive**	**Interpretation**
Source association	8/8	8/8	KPI page points to the supporting source file/region.
KPI identity supported by source	7/8	6/8	One disagreement concerned the generic `Ratio` helper; `Variance Type` was unanimously unsupported.
Source anchor/region	8/8	8/8	Reported source anchor materially aligns with the reviewed region.
Declared-formula origin	8/8	8/8	Absence/presence of declared formula evidence was represented honestly.
Executable-evidence origin	8/8	7/8	`Variance Type` misrepresented multi-branch categorical behavior.
Business-metadata origin honesty	8/8	0/8	Source-absent domain, owner, confidence/readiness, and executive interpretation were presented too strongly.
Review status/reason	8/8	8/8	`Needs Review` and owner-confirmation rationale were visible and defensible.
Provenance/lineage	8/8	8/8	Record ID, source, and lineage remained internally consistent.
Published role separation	8/8	0/8	"Certified"/authoritative presentation and default metadata blurred unresolved evidence status.
Review/edit controls	8/8	8/8	Review Queue and record editing controls were visible and understandable.

6. Discussion

6.1. Reconstruction must preserve the requested calculation

The source-based diagnoses explain why expression presence is a weak description of reconstruction success. The original engine removes declarations before selecting mathematical text, and that preprocessing also removes executable string operands. Its selector then chooses one plausible expression from a region that can contain several measures. The observed outputs show two separate obligations: preserve executable tokens during evidence separation and bind the selected expression to the requested KPI output. The four OEE-related cases make the second obligation concrete because the same availability ratio is returned for four distinct reference identities.

Formula recovery research has already established the need to evaluate detected expressions for correctness and completeness [2]. LEAP adds evidence about named performance measures reconstructed into business-facing records. The recorded control-scope and multi-output examples explain which source constructs defeat its original selection procedure. They also distinguish an analyzer's supplied context from the wider behavior represented in the reference: six helper-call cases require information beyond their input snippet. A reconstruction contract should identify both the target value and the source dependencies that the method actually examines.

6.2. Publication needs field-specific evidence

The six correctly identified promoted KPI pages all receive partial overall judgments. Every reviewed page retains source links and visible review status, and every page fails the business-origin and role-separation checks. The problem lies in the strength of the published business assertions: source-absent owners, domains and authoritative wording appear alongside an unresolved review state. The rendering pathway therefore requires evidence decisions for individual fields, including an explicit representation of unknown business information.

Sodalite's link-state checks and its stated need to compare text with code provide a close predecessor [5]. The present audit supplies the KPI-specific field evidence: intact source association can be accompanied by unsupported business metadata within the very same page. Code histories likewise measures whether its record captures the intended source events [6]. Here, the reference objects are calculations and published assertions, so the evaluation traces each observation to the corresponding source region or field origin.

6.3. Occurrence localization changes the measured task

The second-stage cross-system results measure strict native-coordinate compliance after identity acceptance. A baseline can identify a named KPI function while its definition anchor falls outside a narrower reference return span. The exact `current_ratio` example and the uniform function-location diagnostic show why zero primary baseline matches cannot be interpreted as absence of function-level KPI candidates. Evaluation reports should specify whether the target is a function, an assignment, a returned value or another source occurrence, and define how each method's output is mapped to that unit.

The upgraded extractor's broader review routing returns 305 primary matches alongside 7,802 unmatched scored candidates. This operating point exposes substantial candidate-review work; it produces no automatic promotions. The retained 97 expressions and 300 matched cards identify what evidence is available for subsequent inspection. The technical contribution of the record is that these states remain visible, permitting discovery, expression recovery and publication to be examined at their executed boundaries.

7. Limitations and Threats to Validity

The original 40-file challenge and the second-stage 355-file corpus are targeted frozen public-source samples. The 65 resolved original occurrences are Python; the larger corpus is Python-only. The 54 formula rows represent 35 distinct path/snippet inputs across 12 files, and several KPI occurrences share an input. Counts describe these occurrences rather than independent implementations. The formula capture is reproduced exactly through a source-snippet invocation; the historical capture-driver command was not recovered. Eleven imported-call cases are excluded from the original local audit, and six included helper-call cases have source-oracle dependencies outside their supplied snippets.

The original formula results are recorded local-oracle judgments. The rationales treat symbolic dependencies unevenly: a retained helper call is partial because its exception is not expanded, while some retained property references are accepted as correct because the property carries the behavior. The five correct classifications therefore reflect that recorded rubric. The formula review judges native expression text; reproduced rendered formulas constitute a separate diagnostic observation. Only the 11 primary-verdict disagreements received joint dimension reconciliation, so the five-dimension counts retain the original independent responses. Syntactic presence and the `no_formula` label differ. The eight-page audit covers the complete automatic-promotion population of that challenge, with the supplied reviewer credentials and blinding attestations.

The second-stage reference begins with three AI-generated annotations and author-approved reconciliation. Their metadata records exposure to historical project summaries. Source semantics remain unresolved for 703 of 723 references; the other 20 have a confirmed local expression with semantic approval still pending. The original adapter executes candidate extraction

without the original downstream formula renderer. Its optional comment-parser import is disabled by standalone loading; the corpus contains no `Formula:` marker lines. The accepted second-stage formula and card assessments are author-adjudicated technical recommendations. All 97 native expressions and all 300 complete-page outcomes remain unresolved. The original matched publication HTML is unavailable, and five matched upgraded records fail initial rendering.

R01 contains 509 of 723 second-stage positive references, and the paired combined-F1 interval includes zero. Repository resampling uses four clusters for the original study and ten for the second. Historical original precision resampling assigned zero to undefined draws; primary precision is consequently reported as a point estimate, with the archived diagnostics retained. The two studies use different matching rules. Within the second stage, candidate units remain non-equivalent across systems, and no harmonized function-level detection endpoint was evaluated. Cross-system score differences therefore support native-coordinate and review-load interpretation rather than a ranking of function-level detection effectiveness. The post hoc function-location analysis documents geometric relationships and does not change frozen identity decisions or scores. The 73 unresolved second-stage regions retain their prescribed scenarios. Source-derived identifiers locate snapshot occurrences; reviewer effort, maintenance-task outcomes, industrial prevalence and cross-language fidelity were not measured.

8. Conclusion

LEAP provides an inspectable representation for reconstructing KPI documentation from separately originating source, formula and business evidence. The original challenge recovers six of 65 reference occurrences automatically and falsely promotes two records. Reference-directed review classifies five of 54 formula cases as correct, and all eight promoted pages retain their source links while failing business-field origin checks. Exact replay connects the formula outcomes to removal of executable string operands, selection of an intermediate or another KPI's expression, and omission of control context. The second-stage study characterizes a broader review operating point and demonstrates the influence of source-location granularity on occurrence matching. Together, the results establish concrete verification obligations at source preparation, output binding and field-level publication, supported by preserved source records and reproducible diagnostic analyses.

Related-study evidence

A companion explanation-fidelity study reuses the Ethylene Yield and Gross Margin artifacts to examine source-role accuracy, executable-condition preservation, disagreement preservation, and review-authority consistency. The present article reports the source-to-record architecture and its discovery, formula, and external-page evaluations. The shared demonstration artifacts are identified explicitly so that the evidence populations can be compared across the studies.

Data and software availability

Supplementary file S1 (`S1_Results.zip`) contains the accepted formula/page outcomes, acceptance receipts, statistics and claim-to-evidence records. Supplementary file S2 (`S2_Reproduction.zip`) contains the original analysis entry points, supplied source/context corpus, frozen relevant software sources and earlier experiment evidence. Their captions and integrity manifests accompany the submission files. The complete original frozen-measurements archive is retained unchanged. Public results and scoring scripts are available at `https://github.com/Meshaal-Mouawad/LEAP-Paper1-Evidence`, commit `446dde42ad63433091806bfde4acc0afb55023fa`. This repository contains the screened public subset. Supplement S3 (`S3_Diagnostic_Reproduction.zip`) supplies the new source-based analyses, de-identified review-row projections, original formula-component source and replay instructions. Additional retained research materials are available from the author upon reasonable request.

CRediT authorship contribution statement

Meshaal Obaid Mouawad: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Visualization, Writing – original draft, Writing – review & editing, Project administration.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The author declares that he has no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During preparation of this manuscript, the author used OpenAI Codex to assist with organizing recorded evidence, drafting and revising text, and preparing

LaTeX and submission artifacts. The source-based technical assessment and author adjudication are described in the Methods. The author retains responsibility for the content; no independent prospective reviews are attributed to the tool.

A. Exact source-occurrence identity

To construct the hash input, LEAP converts the anchor to decimal text and joins the three fields with the literal single character `|`. The resulting UTF-8 byte string is therefore exactly `relative_path + "|" + str(anchor) + "|" + normalized_name`. Let s denote that joined string. SHA-256 hashes its UTF-8 encoding. Let h denote the resulting lowercase hexadecimal digest, which has 64 characters. The identifier takes the first 16 characters of h, converts them to uppercase, and attaches the fixed prefix `KPIREC-`. In Eq. 4, $\|$ denotes concatenation of that prefix and the final 16-character string:

$$\begin{aligned} h &= \mathrm{hex}(\mathrm{SHA256}(\mathrm{UTF8}(s)))\,, \\ \mathrm{ID} &= \texttt{KPIREC-} \;\|\; \mathrm{uppercase}(h_{1:16}). \end{aligned} \tag{4}$$

An unchanged relative path, anchor, and normalized name reproduce the identifier even when the repository is checked out at a different absolute location. Moving the file, inserting lines before the anchor, or renaming the KPI changes an input and can change the identifier. Editing only the formula body can leave it unchanged, because formula content is not an identifier input. A separate source-content hash detects whether the bound evidence has changed. These two values support reproducible record location and source freshness, respectively; they are not a mechanism for tracking a logical KPI across revisions.

B. Implementation verification

B.1. Claims and falsification conditions

These scopes establish where LEAP was tested. The next step is to specify what outcome would count against each architectural claim. Table 14 defines these claims in testable terms. Each claim has a concrete condition that would fail the claim in the evaluated implementation.

B.2. Verification layers

Those claims require different evidence: implementation checks can establish that a field is transported, whereas source annotation and independent review are needed to evaluate whether the field is correct. The focused regression suite contains 14 tests for evidence-channel separation, source-derived identity, deterministic traversal, multi-KPI handling, and held-out guardrails. The prepared-record scenario suite contains 38 fixtures covering clean cases, formula/source conflicts, technical markers, rule mappings, corresponding clean controls, combined signals, and formula-shaped presentation inputs. Publication contracts inspect generated RST sources: 12 checks cover evidence preservation in the curated Gross Margin and Ethylene KPI pages, and five checks cover six-file held-out publication identity.

The external challenge adds a separate measurement layer because software tests alone cannot establish which KPI occurrences should be found. File-level annotation agreement was measured with Cohen's κ for pairs of annotators and Fleiss' κ for three annotators; both account for agreement expected by chance. Experiment 1 is retained as exploratory replication evidence because its three-volunteer file-level Fleiss κ was 0.202 and provenance metadata required clarification; it is not used as the primary scoring oracle. Without changing the corpus or annotation definition, Experiment 2 used three new volunteers with pre-annotation blinding attestations. Pairwise file-level Cohen κ values were 0.533, 0.689, and 0.570; three-volunteer Fleiss κ was 0.596, with 32/40 file labels unanimous before reconciliation. Reconciliation retained 65 KPI instances, preserved two KPI identities as unresolved, and produced 54 locally resolved executable-formula oracle cases plus 11 locally unresolved imported-call cases. The exact source files were materialized and checked against their frozen Git blob SHAs before execution.

Table 15 summarizes the verification layers and their recorded outcomes.

C. Detailed second-stage evidence

C.1. Frozen native-coordinate scoring and uncertainty

The prospective evaluation used ten frozen repositories and 355 eligible Python files. The source reference contained 723 adopted positive source occurrences and 73 unresolved regions. All systems were evaluated against the same positive reference using their native candidate units. The common identity and coordinate predicates measure strict native-output-to-reference compliance; the baselines' function-definition anchors and the references' internal calculation spans are non-equivalent localization units. The accepted identity decisions comprise 1,627 candidate/reference pairs; these are possible identity relationships, not a true-positive count. The unchanged scorer applied the frozen coordinate rule and a maximum-cardinality one-to-one allocation before computing each operating point. Precision is the proportion of scored predictions matched to positive references; recall is the proportion of positive references matched; F1 is their harmonic mean. Empty denominators remain undefined. The four source-overlapping upgraded candidates excluded from primary review/combined scoring remain accounted for in the unresolved scenarios.

Original LEAP emitted 19 automatic and 354 review candidates. Upgraded LEAP emitted 8,111

Table 14
Major claims, evidence, and falsification conditions.

Claim	Evidence used	Falsification condition
Evidence-role and source-link retention	Curated source-to-record generation; Ethylene KPI page; publication contract	A required evidence role or source relationship is lost between the stored record and the published page.
Disagreement preservation	Gross Margin declared/executable formulas, source provenance, review state	One formula representation overwrites or hides the other, or publication loses the conflict and its source relationship.
Deterministic rebuilding	Repeated extraction payload hashes and reStructuredText (RST) publication-tree hashes	Identical inputs and configuration produce different ordered extraction payloads or publication trees.
Held-out reconstruction and identity	Six external files; two promoted Overall Equipment Effectiveness (OEE) records; seven review-routed candidates; source-derived record IDs	Same-named calculations from different files collapse into one record, source identities change across repeated runs, or publication loses the source distinction.
Source-to-record provenance binding	Prepared record, source path/anchor, origin-labeled formula channels, publication contract	A published evidence item is attached to the wrong reconstructed record or loses the source/origin relationship required by the evaluated contract.
Complete external KPI page fidelity	Independent review of all eight auto-promoted external KPI pages	A source-absent business fact is presented as established evidence, a promoted identity is unsupported by source, or publication language contradicts the unresolved review state.

review candidates and no automatic predictions. The keyword/regex and structural baselines emitted 96 and 91 automatic predictions, respectively. Under strict native-coordinate combined allocation, original LEAP matched three positive references and upgraded LEAP matched 305; neither baseline satisfied the primary matching rule. The upgraded automatic endpoint had TP 0, FP 0 and FN 723, with zero recall and F1 and undefined precision. Under this strict endpoint, its automatic F1 difference from the structural baseline was zero, with paired 95% interval [0, 0]; the predefined positive-improvement criterion was not met.

The combined upgraded-minus-original F1 difference under strict native-coordinate scoring was 6.361 percentage points, with paired repository-bootstrap 95% interval [-0.046, 7.286] percentage points. This observed difference is not statistically established by an interval that includes zero. Intervals used 10,000 repository resamples with replacement, seed 20261028, pooling counts within each resample. All ten repositories, including zero-positive repositories, remained in the resampling inventory. Full precision, recall and F1 intervals and undefined-replicate counts accompany the results.

R01 held 509 of 723 positive references (70.4%) and contributed 258 of the 305 upgraded combined matches (84.6%). Excluding R01 left 47 TP, 1,891 FP and 167 FN: precision 2.425%, recall 21.963% and F1 4.368%. The repository table and leave-one-repository-out results therefore accompany the pooled native-coordinate results. This identifies the concentration of the observed result rather than attributing a general performance improvement to the larger candidate set.

Matching sensitivity changed upgraded combined TP from 305 under the primary anchor-in-span rule to 273 under exact-start matching and 374 under span overlap. Original LEAP retained three matches and both baselines retained zero under all three rules. The prescribed all-unresolved-negative scenario gave upgraded TP 305, FP 7,806 and FN 418; the all-unresolved-positive scenario gave TP 307, FP 7,804 and FN 489. These two scenarios retain affected candidates under the two prescribed assignments.

Table 16 reports all primary native-coordinate operating points.

Table 17 gives the repository confusion counts.

Table 18 reports the combined native-coordinate intervals.

C.2. Prospective formula availability and adjudicated outcomes

Formula assessment retains all 723 positive references for each LEAP system, giving 1,446 accepted system/reference bindings. Original LEAP had no retained formula output in any of its 723 bound cases. Upgraded LEAP had 418 reference occurrences unmatched under the primary rule, 208 matched outputs retaining evidence without an expression, and 97 matched native expressions. Output absence therefore remains visible in the full denominator rather than being removed before formula assessment.

Table 15
Verification and challenge-evaluation layers.

Evidence layer	Unit	Outcome
Focused correction/guardrail regression suite	14 test functions	14/14 passed
Prepared-record scenario suite	38 fixtures	38/38 passed
Curated extraction determinism	5 repeated scans	Identical extraction payload hashes
Curated publication determinism	4 repeated generations	Identical RST-tree hashes
Six-file held-out extraction determinism	5 repeated scans	Identical extraction payload hashes
Six-file held-out publication determinism	3 repeated generations	Identical RST-tree hashes
Curated publication contract	12 checks	12/12 passed
Held-out publication identity contract	5 checks	5/5 passed
Experiment 2 independent source labeling	3 volunteers, 40 files	32/40 unanimous; Fleiss $\kappa = 0.5958$ before reconciliation
Experiment 2 reconciliation	67 candidate identities	65 resolved KPIs; 2 unresolved identities preserved outside scoring
Formula-fidelity independent review and reconciliation	54 resolved cases, 2 volunteers	43/54 exact verdict agreement, Cohen $\kappa = 0.6035$; 11 verdict disagreements reconciled
External complete-record fidelity	8 promoted KPI pages, 2 volunteers	8/8 identical overall verdicts, Cohen $\kappa = 1.0$; 6 partially correct, 2 incorrect; 7/7 navigation checks passed by both
Challenge promotion comparison	LEAP + 2 baselines	TP/FP/FN, precision, recall, F1, repository-bootstrap intervals, scorer sensitivity, per-repository and leave-one-repository-out diagnostics

Table 16
Second-stage native output, strict occurrence-coordinate scoring. TP/FP/FN apply the frozen identity and coordinate predicates to non-equivalent native candidate units. Precision, recall and F1 are percentages; – denotes undefined precision. Review and automatic populations remain separate.

System	Channel	TP	FP	FN	Precision	Recall	F1
Original LEAP	Automatic	1	18	722	5.263	0.138	0.270
Original LEAP	Review	2	352	721	0.565	0.277	0.371
Original LEAP	Combined	3	370	720	0.804	0.415	0.547
Upgraded LEAP	Automatic	0	0	723	–	0.000	0.000
Upgraded LEAP	Review	305	7802	418	3.762	42.185	6.908
Upgraded LEAP	Combined	305	7802	418	3.762	42.185	6.908
Keyword/regex	Automatic	0	96	723	0.000	0.000	0.000
Keyword/regex	Review	0	0	723	–	0.000	0.000
Keyword/regex	Combined	0	96	723	0.000	0.000	0.000
Structural	Automatic	0	91	723	0.000	0.000	0.000
Structural	Review	0	0	723	–	0.000	0.000
Structural	Combined	0	91	723	0.000	0.000	0.000

The source oracle contained 20 locally confirmed expressions and 703 semantically unresolved cases. Among the 20, upgraded LEAP supplied one matched native expression, four evidence-only matches and 15 reference occurrences unmatched under the primary rule. The remaining 96 native-expression cases intersected unresolved source semantics. Native-expression presence and analyzer COMPLETE, PARTIAL or UNRESOLVED states describe retained output availability and analysis state.

For P1P00084, the matched native expression before - after corresponds to the source-local assignment change = before - after. The retained surrounding context includes a None guard and a noise-floor-dependent return. The local expression corresponds to the assignment; the surrounding guard and return remain part of the unresolved semantic assessment. The second-stage seven-axis rubric separately assesses operands/operators, guards/exceptions, aggregation/window behavior, units/scaling, rounding/clipping, null/default behavior

Table 17
Primary combined native-coordinate results by repository. Each system column gives TP/FP/FN under the strict occurrence-coordinate rule. Zero-positive repositories remain in the inventory.

Repository	Positive gold	Original	Upgraded	Regex	Structural
R01	509	1/218/508	258/5911/251	0/63/509	0/61/509
R02	30	2/8/28	11/154/19	0/0/30	0/0/30
R03	1	0/22/1	1/255/0	0/0/1	0/0/1
R04	6	0/7/6	3/71/3	0/0/6	0/0/6
R05	0	0/2/0	0/5/0	0/0/0	0/0/0
R06	0	0/0/0	0/0/0	0/0/0	0/0/0
R07	0	0/1/0	0/0/0	0/0/0	0/0/0
R08	1	0/26/1	0/560/1	0/3/1	0/1/1
R09	132	0/52/132	27/614/105	0/7/132	0/6/132
R10	44	0/34/44	5/232/39	0/23/44	0/23/44

Table 18
Combined strict native-coordinate 95% repository-bootstrap intervals, in percentage units. All-channel intervals, undefined-replicate counts and leave-one-repository-out results are retained in the accompanying evidence tables.

System	Precision	Recall	F1
Original LEAP	[0.000, 4.000]	[0.000, 4.226]	[0.000, 3.670]
Upgraded LEAP	[0.942, 4.238]	[17.587, 50.479]	[1.810, 7.730]
Keyword/regex	[0.000, 0.000]	[0.000, 0.000]	[0.000, 0.000]
Structural	[0.000, 0.000]	[0.000, 0.000]	[0.000, 0.000]

and interprocedural dependencies. The earlier component audit retains its separate 54-case classification distribution.

The accepted overall formula classifications were 723 NO_FORMULA cases for original LEAP, and 626 NO_FORMULA plus 97 UNRESOLVED cases for upgraded LEAP (Table 19). Neither system received a CORRECT, PARTIAL or INCORRECT overall classification in this prospective assessment. The 97 unresolved outcomes retain source-semantic and scope questions. The single locally supported native-expression case, P1P00084, also remains UNRESOLVED overall after acceptance.

The documented-correct fraction is 0/723 for both systems. Among adjudicable cases it is 0/723 for original and 0/626 for upgraded LEAP. The paired table contains 626 NO_FORMULA/NO_FORMULA pairs and 97 original-NO_FORMULA/upgraded-UNRESOLVED pairs. Thus the jointly adjudicable comparison contains 626 cases and has a zero difference in documented correctness. The 97 unresolved upgraded outputs remain outside that paired adjudicable population. Preserving those unknowns gives an upgraded correctness identification range of 0–13.416% over all 723 cases; original LEAP's range is 0% because its formula outputs are absent.

Upgraded NO_FORMULA prevalence is 86.584%, with repository-bootstrap 95% interval [69.444, 97.159]%; UNRESOLVED prevalence is 13.416%, with interval [2.841, 30.556]%. Bootstrap documented-correct fractions remain zero in every defined resample because no case is classified CORRECT. Full accepted counts, paired identities and repository summaries accompany the manuscript.

C.3. Prospective review-card publication evidence

The upgraded initial rendering population contained 7,873 initially successful review cards, 196 initial renderer failures and 42 cards rendered only in a separate diagnostic pass. The diagnostic cards remain separate and do not count as successful initial upgraded rendering. Under accepted primary matching, 300 of the upgraded system's 305 matched review records had initially successful cards, while five selected records were rejected by the initial upgraded renderer. The three original LEAP matched outputs had no available publication HTML.

The 300 matched cards were read from the frozen archives. Archive, HTML-member and card identities were verified, and their rendered source paths and anchors were compared with the bound source files. The evidence inventory retains field labels, formula presentation, review summaries, controls and published wording. The source-association check verifies that the inspected card points to the expected source occurrence. Content judgments are recorded field by field. The cards belong to the review population, not the automatically promoted complete-page population. The upgraded

Table 19
Author-adjudicated prospective formula classifications. Each system retains 723 gold cases.

System	Correct	Partial	Incorrect	Unresolved	No formula	Adjudicable
Original LEAP	0	0	0	0	723	723
Upgraded LEAP	0	0	0	97	626	626

automatic-TP denominator is zero and its complete-page fidelity rate is undefined. The earlier eight-page correctness audit remains the distinct completed publication experiment.

Owner acceptance assigns all 300 inspected review cards PARTIALLY_CORRECT within their review-card scope, while preserving all 300 proposed complete-page outcomes as UNRESOLVED. Source association, source anchors, accepted KPI identity, and candidate-only review status with retained native review checks are classified CORRECT for all 300. Provenance/lineage and review/correction controls are PARTIAL for all 300: evidence navigation is retained, but no explicit edit or review-queue control is present on these cards, and linked evidence does not alone settle complete lineage truth. Business-information origin remains UNRESOLVED for all 300.

No declared formula is retained on these cards. Executable-formula origin and formula wording are UNRESOLVED for the 97 expression-bearing records and NOT_APPLICABLE for the 203 without an expression. Six card texts retain wording tokens requiring contextual interpretation; their accepted wording component remains UNRESOLVED. The other 294 have no explicit authoritative claim in the assessed card text.

The complete-page adjudicable denominator is zero, so no complete-page correctness rate or confidence interval is computed. The upgraded automatic-TP denominator remains zero. The five matched renderer failures, 196 initial upgraded rendering failures, 42 diagnostic-only cards and unavailable original publication outputs retain their earlier dispositions.

References

[1] M. Moser, J. Pichler, G. Fleck, M. Witlatschil, RbG: A documentation generator for scientific and engineering software, in: 2015 IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER), IEEE, 2015, pp. 464–468. https://doi.org/10.1109/SANER.2015.7081857.

[2] O. Moseler, F. Lemmer, S. Baltes, S. Diehl, On the diversity and frequency of code related to mathematical formulas in real-world Java projects, Journal of Systems and Software 172 (2021) 110863. https://doi.org/10.1016/j.jss.2020.110863.

[3] A. Maté, J. Trujillo, J. Mylopoulos, Specification and derivation of key performance indicators for business analytics: A semantic approach, Data & Knowledge Engineering 108 (2017) 30–49. https://doi.org/10.1016/j.datak.2016.12.004.

[4] M. del Mar Roldán-García, J. García-Nieto, A. Maté, J. Trujillo, J.F. Aldana-Montes, Ontology-driven approach for KPI meta-modelling, selection and reasoning, International Journal of Information Management 58 (2021) 102018. https://doi.org/10.1016/j.ijinfomgt.2019.10.003.

[5] A. Horvath, A. Macvean, B.A. Myers, Support for long-form documentation authoring and maintenance, in: 2023 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC), IEEE, 2023, pp. 109–114. https://doi.org/10.1109/VL-HCC57772.2023.00020.

[6] V.T.T. Pham, C. Kelleher, Code histories: Documenting development by recording code influences and changes in code, Journal of Computer Languages 82 (2025) 101313. https://doi.org/10.1016/j.cola.2024.101313.

[7] D.E. Knuth, Literate programming, The Computer Journal 27 (2) (1984) 97–111. https://doi.org/10.1093/comjnl/27.2.97.

[8] K. Nørmark, Elucidative programming, Nordic Journal of Computing 7 (2) (2000) 87–105. https://homes.cs.aau.dk/~normark/elucidative-programming/papers/njc-paper.pdf.

[9] J.F. Pimentel, L. Murta, V. Braganholo, J. Freire, A large-scale study about quality and reproducibility of Jupyter notebooks, in: 2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR), 2019, pp. 507–517. https://doi.org/10.1109/MSR.2019.00077.

[10] J. Carette, S.W. Smith, J. Balaci, Generating software for well-understood domains, in: Eelco Visser Commemorative Symposium (EVCS 2023), OASIcs 109, 2023, pp. 7:1–7:12. https://doi.org/10.4230/OASIcs.EVCS.2023.7.

[11] R. Husák, J. Kofroň, F. Zavoral, Combining static analysis techniques for program comprehension using Slicito, in: 2025 IEEE/ACM 33rd International Conference on Program Comprehension (ICPC), IEEE, 2025. https://doi.org/10.1109/ICPC66645.2025.00048.

[12] M. De Sanctis, L. Iovino, M.T. Rossi, M. Wimmer, MIKADO: a smart city KPIs assessment modeling framework, Software and Systems Modeling 21 (2022) 281–309. https://doi.org/10.1007/s10270-021-00907-9.

[13] E. Domínguez, B. Pérez, A.L. Rubio, M.A. Zapata, A flexible framework to ensure traceability, consistency, and propagation of KPIs evolution, Journal of Software: Evolution and Process 37 (2) (2025) e70004. https://doi.org/10.1002/smr.70004.

[14] C. Diamantini, T. Khan, D. Potena, E. Storti, Semantic models of performance indicators: A systematic survey, ACM Computing Surveys 57 (8) (2025) 202:1–202:37. https://doi.org/10.1145/3719291.

[15] R. Pérez-Castillo, I. García Rodríguez de Guzmán, M. Piattini, Business process archeology using MARBLE, Information and Software Technology 53 (10) (2011) 1023–1044. https://doi.org/10.1016/j.infsof.2011.05.006.

[16] P.K. Chittimalli, A. Bhattacharyya, SBVR-based business rule creation for legacy programs using variable provenance, in: Proceedings of the 12th Innovations on Software Engineering Conference (ISEC '19), ACM, 2019, pp. 16:1–16:11. https://doi.org/10.1145/3299771.3299786.

[17] A. Marcus, J.I. Maletic, A. Sergeyev, Recovery of traceability links between software documentation and source code, International Journal of Software Engineering and Knowledge Engineering 15 (5) (2005) 811–836. https:

//doi.org/10.1142/S0218194005002543.

[18] L. Moreau, P. Missier (eds.), PROV-DM: The PROV Data Model, W3C Recommendation, 30 April 2013. https://www.w3.org/TR/prov-dm/.

[19] Nguyen The-Vinh, Policy-aware interface contracts for testing access-control and selected privacy obligations in service APIs, Journal of Computer Languages 89 (2026) 101424. https://doi.org/10.1016/j.cola.2026.101424.